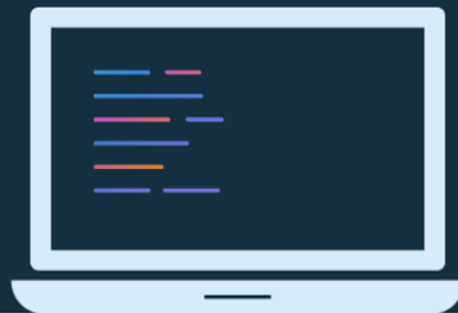




Android Development with Kotlin



Data binding

Current approach: findViewById()

Traverses the `View` hierarchy each time

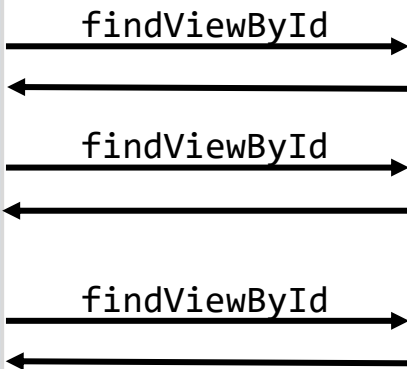
MainActivity.kt

```
val name = findViewById(...)
val age = findViewById(...)
val loc = findViewById(...)

name.text = ...
age.text = ...
loc.text = ...
```

activity_main.xml

```
<ConstraintLayout ... >
  <TextView
    android:id="@+id/name"/>
  <TextView
    android:id="@+id/age"/>
  <TextView
    android:id="@+id/loc"/>
</ConstraintLayout>
```



Use data binding instead

Bind UI components in your layouts to data sources in your app.

MainActivity.kt

```
Val binding:ActivityMainBinding
```

```
binding.name.text = ...
```

```
binding.age.text = ...
```

```
binding.loc.text = ...
```

initialize binding

activity_main.xml

```
<layout>
  <ConstraintLayout ... >
    <TextView
      android:id="@+id/name"/>
    <TextView
      android:id="@+id/age"/>
    <TextView
      android:id="@+id/loc"/>
  </ConstraintLayout>
</layout>
```

Modify build.gradle file

```
android {  
    ...  
    buildFeatures {  
        dataBinding true  
    }  
}
```

Add layout tag

<layout>

```
<androidx.constraintlayout.widget.ConstraintLayout>  
    <TextView ... android:id="@+id/username" />  
    <EditText ... android:id="@+id/password" />  
</androidx.constraintlayout.widget.ConstraintLayout>
```

</layout>

Layout inflation with data binding

Replace this

```
setContentView(R.layout.activity_main)
```

with this

```
val binding: ActivityMainBinding = DataBindingUtil.setContentView(  
    this, R.layout.activity_main)
```

```
binding.username = "Melissa"
```

Data binding layout variables

```
<layout>
    <data>
        <variable name="name" type="String"/>
    </data>
    <androidx.constraintlayout.widget.ConstraintLayout>
        <TextView
            android:id="@+id/textView"
            android:text="@{name}" />
        </androidx.constraintlayout.widget.ConstraintLayout>
</layout>
```

In MainActivity.kt:

```
binding.name = "John"
```


Data binding layout expressions

```
<layout>
  <data>
    <variable name="name" type="String"/>
  </data>

  <androidx.constraintlayout.widget.ConstraintLayout>
    <TextView
      android:id="@+id/textView"
      android:text="@{name.toUpperCase()}" />
    </androidx.constraintlayout.widget.ConstraintLayout>
</layout>
```

Android app architecture

Avoid short-term hacks

- External factors, such as tight deadlines, can lead to poor decisions about app design and structure.
- Decisions have consequences for future work (app can be harder to maintain long-term).
- Need to balance on-time delivery and future maintenance burden.

Examples of short-term hacks

- Tailoring your app to a specific device
- Blindly copying and pasting code into your files
- Placing all business logic in activity file
- Hardcoding user-facing strings in your code

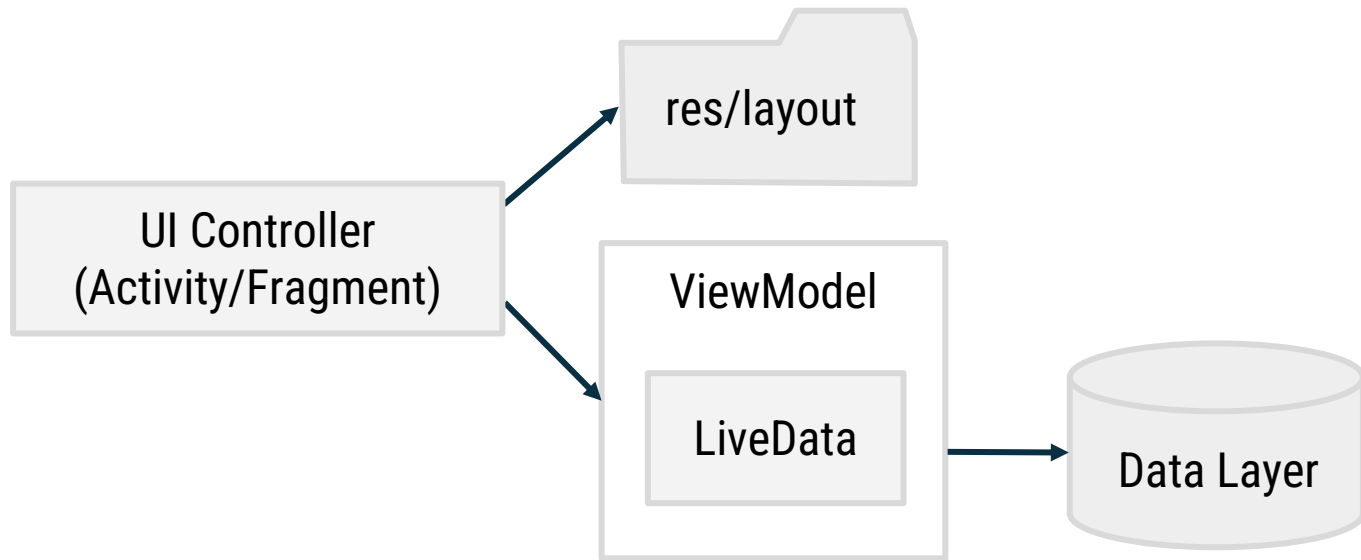
Why you need good app architecture

- Clearly defines where specific business logic belongs
- Makes it easier for developers to collaborate
- Makes your code easier to test
- Lets you benefit from already-solved problems
- Saves time and reduces technical debt as you extend your app

Android Jetpack

- Android libraries that incorporate best practices and provide backward compatibility in your apps
- Jetpack comprises the `androidx.*` package libraries

Separation of concerns



Architecture components

- Architecture design patterns, like MVVM and MVI, describe a loose template for what the structure of your app should be.
- Jetpack architecture components help you design *robust*, *testable*, and *maintainable* apps.

ViewModel

Gradle: lifecycle extensions

In `app/build.gradle` file:

```
dependencies {  
    implementation "androidx.lifecycle:lifecycle-viewmodel-ktx:$lifecycle_version"  
    implementation "androidx.activity:activity-ktx:$activity_version"  
}
```

Gradle: lifecycle extensions (New)

[versions]

lifecycleViewModelKtx = "2.6.1"

libs.versions.toml (Version Catalog)

[libraries]

androidx-lifecycle-viewmodel-ktx = { group = "androidx.lifecycle",
name = "lifecycle-viewmodel-ktx",
version.ref = "lifecycleViewModelKtx" }

dependencies {

build.gradle.kts (Module: app)

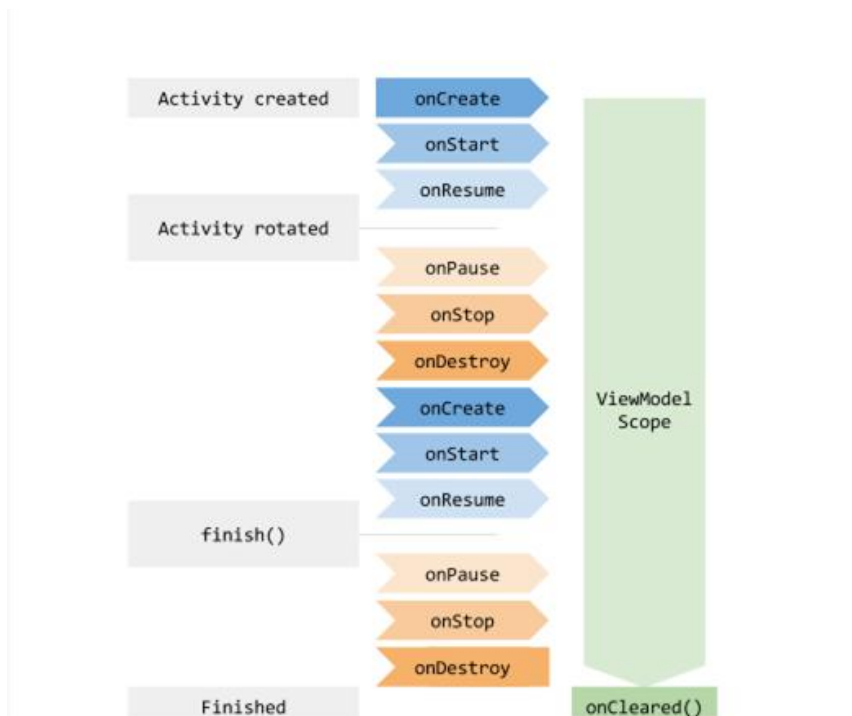
implementation(*libs.androidx.lifecycle.viewmodel.ktx*)

}

ViewModel

- Prepares data for the UI
- Must not reference activity, fragment, or views in view hierarchy
- Scoped to a lifecycle (which activity and fragment have)
- Enables data to survive configuration changes
- Survives as long as the scope is alive

Lifetime of a ViewModel



Kabaddi Kounter



ViewModel class

abstract class ViewModel

Summary

Public constructors

`<init>()`

ViewModel is a class that is responsible for preparing and managing the data for an [Activity](#) or a [Fragment](#).

Protected methods

open [Unit](#)

`onCleared()`

This method will be called when this ViewModel is no longer used and will be destroyed.

Extension properties

From [androidx.lifecycle](#)

[CoroutineScope](#)

`viewModelScope`

[CoroutineScope](#) tied to this [ViewModel](#).

Implement a ViewModel

```
class ScoreViewModel : ViewModel() {  
    var scoreA : Int = 0  
    var scoreB : Int = 0  
    fun incrementScore(isTeamA: Boolean) {  
        if (isTeamA) {  
            scoreA++  
        } else {  
            scoreB++  
        }  
    }  
}
```


Load and use a ViewModel

```
class MainActivity : AppCompatActivity() {  
    // Delegate provided by androidx.activity.viewModels  
    val viewModel: ScoreViewModel by viewModels()  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        ...  
        val scoreViewA: TextView = findViewById(R.id.scoreA)  
        scoreViewA.text = viewModel.scoreA.toString()  
    }  
}
```

Using a ViewModel

Within `MainActivity onCreate()`:

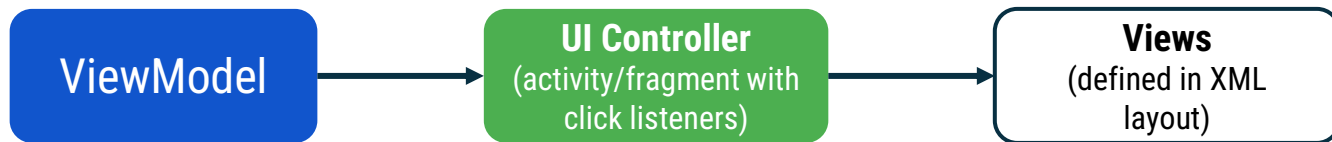
```
val scoreViewA: TextView = findViewById(R.id.scoreA)
val plusOneButtonA: Button = findViewById(R.id.plusOne_teamA)

plusOneButtonA.setOnClickListener {
    viewModel.incrementScore(true)
    scoreViewA.text = viewModel.scoreA.toString()
}
```

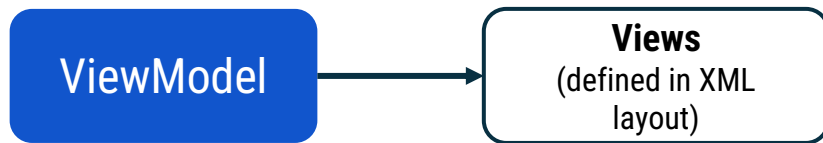
Data binding

ViewModels and data binding

- App architecture without data binding



- ViewModels can work in concert with data binding



Data binding in XML revisited

Specify ViewModels in the `data` tag of a binding.

```
<layout>
  <data>
    <variable>
      name="viewModel"
      type="com.example.kabaddikounter.ScoreViewModel" />
    </data>
  <ConstraintLayout ../>
</layout>
```

Attaching a ViewModel to a data binding

```
class MainActivity : AppCompatActivity() {  
  
    val viewModel: ScoreViewModel by viewModels()  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        val binding: ActivityMainBinding = DataBindingUtil.setContentView(this,  
            R.layout.activity_main)  
  
        binding.viewModel = viewModel  
        ...  
    }  
}
```

Using a ViewModel from a data binding

In `activity_main.xml`:

```
<TextView
    android:id="@+id/scoreViewA"
    android:text="@{viewModel.scoreA.toString()}" />

...
```

ViewModels and data binding

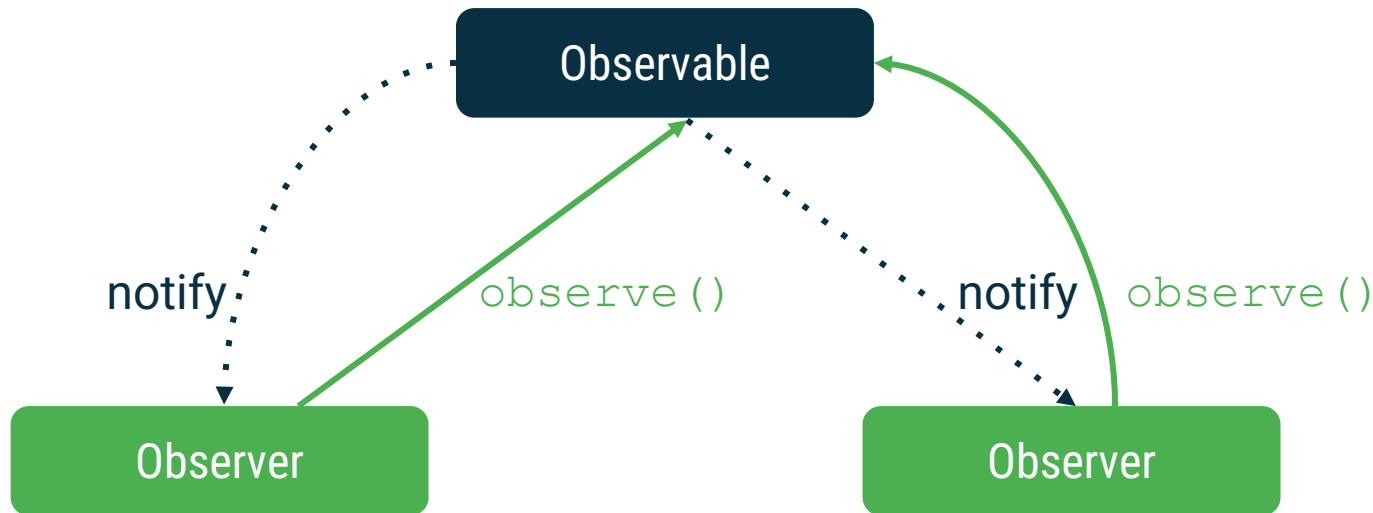
```
override fun onCreate(savedInstanceState: Bundle?) {  
    ...  
    val binding: ActivityMainBinding = DataBindingUtil.setContentView(this,  
        R.layout.activity_main)  
  
    binding.plusOneButtonA.setOnClickListener {  
        viewModel.incrementScore(true)  
        binding.scoreViewA.text = viewModel.scoreA.toString()  
    }  
}
```


LiveData

Observer design pattern

- Subject maintains list of observers to notify when state changes.
- Observers receive state changes from subject and execute appropriate code.
- Observers can be added or removed at any time.

Observer design pattern diagram



LiveData

- A lifecycle-aware data holder that can be observed
- Wrapper that can be used with any data including lists (for example, `LiveData<Int>` holds an `Int`)
- Often used by ViewModels to hold individual data fields
- Observers (activity or fragment) can be added or removed
 - `observe(owner: LifecycleOwner, observer: Observer)`
 - `removeObserver(observer: Observer)`

LiveData versus MutableLiveData

LiveData<T>	MutableLiveData<T>
● <code>getValue()</code>	● <code>getValue()</code> ● <code>postValue(value: T)</code> ● <code>setValue(value: T)</code>

T is the type of data that's stored in `LiveData` or `MutableLiveData`.

Use LiveData in ViewModel

```
class ScoreViewModel : ViewModel() {  
  
    private val _scoreA = MutableLiveData<Int>(0)  
    val scoreA: LiveData<Int>  
        get() = _scoreA  
  
    fun incrementScore(isTeamA: Boolean) {  
        if (isTeamA) {  
            _scoreA.value = _scoreA.value!! + 1  
        }  
        ...  
    }  
}
```

Add an observer on LiveData

Set up click listener to increment `ViewModel` score:

```
binding.plusOneButtonA.setOnClickListener {  
    viewModel.incrementScore(true)  
}
```

Create observer to update team A score on screen:

```
val scoreA_Observer = Observer<Int> { newValue ->  
    binding.scoreViewA.text = newValue.toString()  
}
```

Add the observer onto `scoreA LiveData` in `ViewModel`:

```
viewModel.scoreA.observe(this, scoreA_Observer)
```

Two-way data binding

- We already have two-way binding with `ViewModel` and `LiveData`.
- Binding to `LiveData` in XML eliminates need for an observer in code.

Example layout XML

```
<layout>
  <data>
    <variable>
      name="viewModel"
      type="com.example.kabaddikounter.ScoreViewModel" />
    </data>
    <ConstraintLayout ...>
      <TextView ...
        android:id="@+id/scoreViewA"
        android:text="@{viewModel.scoreA.toString()}" />
      ...
    </ConstraintLayout>
</layout>
```

Example Activity

```
class MainActivity : AppCompatActivity() {  
    val viewModel: ScoreViewModel by viewModels()  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        val binding: ActivityMainBinding = DataBindingUtil  
            .setContentView(this, R.layout.activity_main)  
  
        binding.viewModel = viewModel  
        binding.lifecycleOwner = this  
  
        binding.plusOneButtonA.setOnClickListener {  
            viewModel.incrementScore(true)  
        }  
        ...  
    }  
}
```

Example ViewModel

```
class ScoreViewModel : ViewModel() {  
    private val _scoreA = MutableLiveData<Int>(0)  
    val scoreA : LiveData<Int>  
        get() = _scoreA  
    private val _scoreB = MutableLiveData<Int>(0)  
    val scoreB : LiveData<Int>  
        get() = _scoreB  
    fun incrementScore(isTeamA: Boolean) {  
        if (isTeamA) {  
            _scoreA.value = _scoreA.value!! + 1  
        } else {  
            _scoreB.value = _scoreB.value!! + 1  
        }  
    }  
}
```

Transform LiveData

Manipulating LiveData with transformations

LiveData can be transformed into a new LiveData object.

- `map()`
- `switchMap()`

Example LiveData with transformations

```
val result: LiveData<String> = Transformations.map(viewModel.scoreA) {  
    x -> if (x > 10) "A Wins" else ""  
}
```

Example LiveData with transformations

```
private fun <X, Y> LiveData<X>.map(transform: (X) -> Y): LiveData<Y> {  
    val result = MediatorLiveData<Y>()  
    result.addSource(this) { x ->  
        result.value = transform(x)  
    }  
    return result  
}  
  
val result: LiveData<String> = _scoreA.map { score ->  
    if (score > 10) "A Wins" else ""  
}
```

Android permissions

Permissions

- Protect the privacy of an Android user
- Declared with the `<uses-permission>` tag in the `AndroidManifest.xml`

Permissions granted to your app

- Permissions can be granted during *installation* or *runtime*, depending on the protection level.
- Each permission has a protection level: **normal**, **signature**, or **dangerous**.
- For permissions granted during runtime, prompt users to explicitly *grant* or *deny* access to your app.

Permission protection levels

Protection Level	Granted when?	Must prompt before use?	Examples
Normal	Install time	No	ACCESS_WIFI_STATE, VIBRATE, INTERNET
Signature	Install time	No	N/A
Dangerous	Runtime	Yes	GET_ACCOUNTS, CAMERA, CALL_PHONE, BLUETOOTH

Add permissions to the manifest

In `AndroidManifest.xml`:

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.sampleapp">

    <uses-permission android:name="android.permission.USE_BIOMETRIC" />

    <application>
        <activity
            android:name=".MainActivity" ... >
            ...
        </activity>
    </application>
</manifest>
```

Internet access permissions

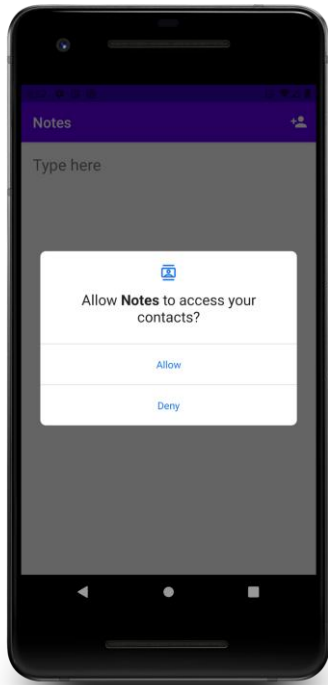
In `AndroidManifest.xml`:

```
<uses-permission android:name="android.permission.INTERNET" />  
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
```

Request dangerous permissions

- Prompt the user to grant permission when they try to access functionality that requires dangerous permission.
- Explain to the user why the permission is needed.
- Fall back gracefully if the user denies the permission (the app should still function).

Prompt for dangerous permission



App permissions best practices

- Only use the permissions necessary for your app to work.
- Pay attention to permissions required by libraries.
- Be transparent.
- Make system accesses explicit.

Asynchronous programming

Long-running tasks

- Download information
- Sync with a server
- Write to a file
- Heavy computation
- Read from, or write to, a database

Need for async programming

- Limited time to do tasks and remain responsive
- Balanced with the need to execute long-running tasks
- Control over how and where tasks are executed

Async programming on Android

- Threading
- Callbacks
- Plus many other options

What is the recommended way?

Coroutines

Coroutines

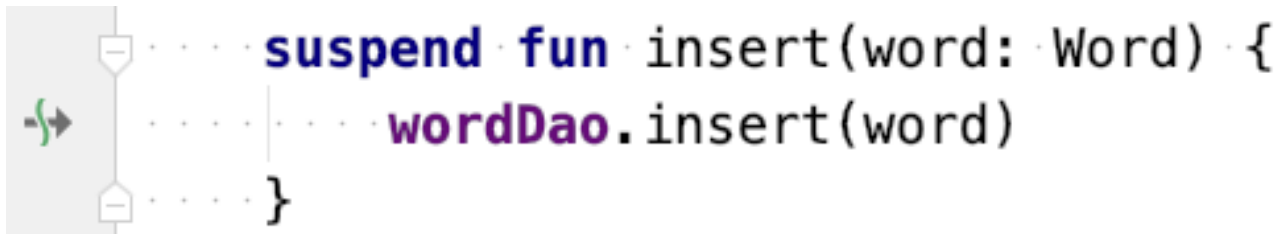
- Keep your app responsive while managing long-running tasks.
- Simplify asynchronous code in your Android app.
- Write code in sequential way
- Handle exceptions with `try/catch` block

Benefits of coroutines

- Lightweight
- Fewer memory leaks
- Built-in cancellation support
- Jetpack integration

Suspend functions

- Add `suspend` modifier
- Must be called by other `suspend` functions or coroutines



```
suspend fun insert(word: Word) {  
    wordDao.insert(word)  
}
```


Suspend and resume

- `suspend`

Pauses execution of current coroutine and saves local variables

- `resume`

Automatically loads saved state and continues execution from the point the code was suspended

Example

```
suspend fun fetchDocs() {  
    val docs = get("...")  
    show(docs)  
}
```

Main Thread
[stack]

suspend

resume



Control where coroutines run

Dispatcher	Description of work	Examples of work
<code>Dispatchers.Main</code>	UI and nonblocking (short) tasks	Updating LiveData, calling suspend functions
<code>Dispatchers.IO</code>	Network and disk tasks	Database, file IO
<code>Dispatchers.Default</code>	CPU intensive	Parsing JSON

withContext

```
suspend fun get(url: String) {  
  
    // Start on Dispatchers.Main  
  
    withContext(Dispatchers.IO) {  
        // Switches to Dispatchers.IO  
        // Perform blocking network IO here  
    }  
  
    // Returns to Dispatchers.Main  
}
```

CoroutineScope

Coroutines must run in a `CoroutineScope`:

- Keeps track of all coroutines started in it (even suspended ones)
- Provides a way to cancel coroutines in a scope
- Provides a bridge between regular functions and coroutines

Examples: `GlobalScope`

`ViewModel` **has** `viewModelScope`

`Lifecycle` **has** `lifecycleScope`

Start new coroutines

- `launch` - no result needed

```
fun loadUI() {  
    launch {  
        fetchDocs()  
    }  
}
```

- `async` - can return a result

ViewModelScope

```
class MyViewModel: ViewModel() {  
  
    init {  
        viewModelScope.launch {  
            // Coroutine that will be canceled  
            // when the ViewModel is cleared  
        }  
    }  
    ...  
}
```

Connect to, and use, network resources

Retrofit

- **Retrofit** is a type-safe REST client for Android, Java, and Kotlin developed by *Square*.
- The library provides a powerful framework for authenticating and interacting with APIs and sending network requests with **OkHttp**.
- (**OkHttp** is a third-party library developed by *Square* for sending and receive HTTP-based network requests, provides an implementation of the **URLConnection** interface)

Retrofit cont'd

- Networking library that turns your HTTP API into a Kotlin and Java interface
- Enables processing of requests and responses into objects for use by your apps
 - Provides base support for parsing common response types, such as XML and JSON
 - Can be extended to support other response types

Retrofit cont'd

- Once the data is downloaded then it is parsed into a Plain Old Java Object (POJO) which must be defined for each "resource" in the response.

Why use Retrofit?

- Builds on industry-standard libraries, like OkHttp, that provide:
 - HTTP/2 support
 - Connection pooling
 - Response caching and enhanced security
- Frees the developer from the scaffolding setup needed to run a request

Add Gradle dependencies

```
implementation "com.squareup.retrofit2:retrofit:2.11.0"  
implementation "com.squareup.retrofit2:converter-moshi:2.11.0"  
  
implementation "com.squareup.moshi:moshi:$moshi_version"  
implementation "com.squareup.moshi:moshi-kotlin:$moshi_version"  
kapt "com.squareup.moshi:moshi-kotlin-codegen:$moshi_version"
```

Add Gradle dependencies

```
implementation "com.google.code.gson:gson:2.11.0"  
implementation "com.squareup.retrofit2:retrofit:2.11.0"  
implementation "com.squareup.retrofit2:converter-gson:2.11.0"  
  
implementation(platform("com.squareup.okhttp3:okhttp-bom:4.10.0"))  
implementation("com.squareup.okhttp3:okhttp")  
implementation("com.squareup.okhttp3:logging-interceptor")
```

Add Gradle dependencies (New)

[versions]

gson = "2.11.0"

retrofit = "2.11.0"

libs.versions.toml (Version Catalog)

[libraries]

gson = { group = "com.google.code.gson",
name = "gson", version.ref = "gson" }

retrofit = { group = "com.squareup.retrofit2",
name = "retrofit", version.ref = "retrofit" }

converter-gson = { group = "com.squareup.retrofit2",
name = "converter-gson", version.ref = "retrofit" }

Add Gradle dependencies (New)

```
dependencies {                                     build.gradle.kts (Module: app)
    implementation(libs.gson)
    implementation(libs.retrofit)
    implementation(libs.converter.gson)
}
```


Connect to a web service

HTTP methods

- GET
- POST
- PUT
- DELETE

Example web service API

URL	DESCRIPTION	METHOD
<code>dummyjson.com/products</code>	Get a list of all products	GET
<code>dummyjson.com/products/id</code>	Get a product by id	GET
<code>dummyjson.com/products/search?q=query</code>	Search products using a filter	GET
<code>dummyjson.com/products/add</code>	Create and add a new product	POST

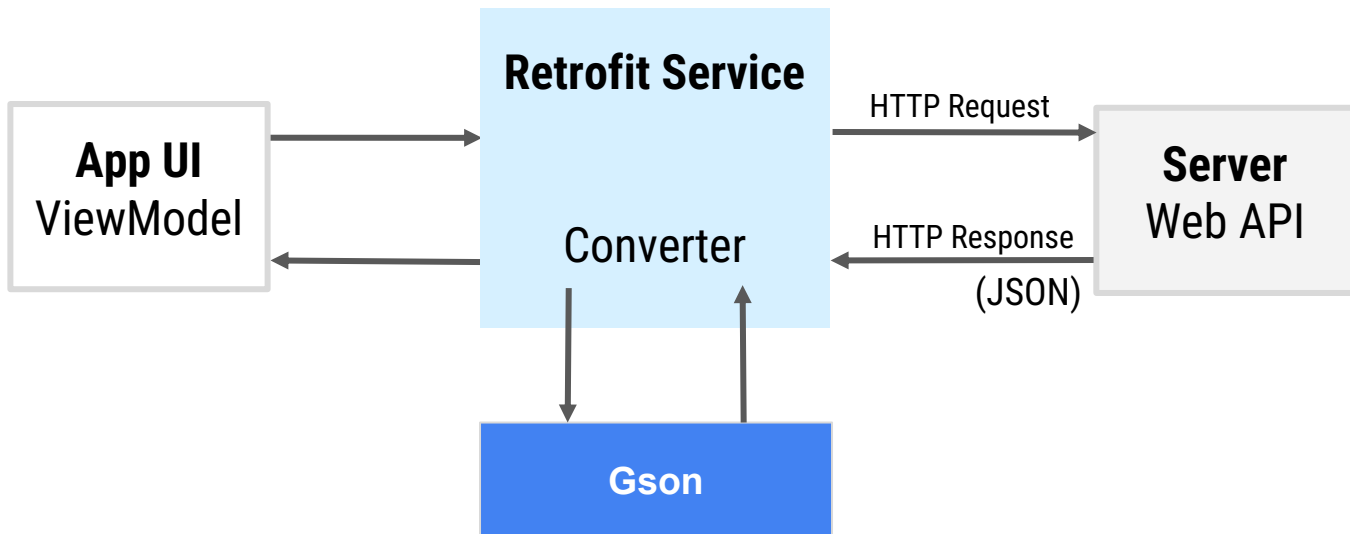
Define a Retrofit service

```
interface SimpleService {  
    @GET("products")  
    suspend fun listAll(): List<Product>  
  
    @GET("products/{productId}")  
    suspend fun getProductBy(@Path("productId") productId: String): Product  
  
    @GET("products/search") // becomes products/search?q=query  
    suspend fun searchProducts(@Query("q") search: String): List<Product>  
  
    @POST("products/add")  
    suspend fun add(@Body product : Product): Product  
}
```

Create a Retrofit object for network access

```
val retrofit = Retrofit.Builder()  
    .baseUrl("https://dummyjson.com")  
    .addConverterFactory(...)  
    .build()  
  
val service = retrofit.create(SimpleService::class.java)
```

End-to-end diagram



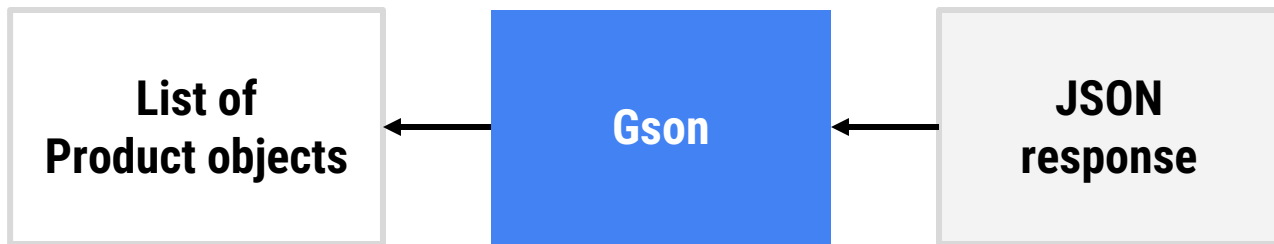
Converter.Factory

Helps convert from a response type into class objects

- JSON (Gson or Moshi)
- XML (Jackson, SimpleXML, JAXB)
- Protocol buffers
- Scalars (primitives, boxed, and Strings)

Gson

- Gson library for parsing JSON into objects and back
- Add Gson library dependencies to your app's Gradle file.
- Configure your Gson builder to use with Retrofit.



JSON code

```
{  
  "id":1,  
  "title":"iPhone 9",  
  "description":"An apple mobile which is nothing like apple",  
  "price":549,  
  "discountPercentage":12.96,  
  "rating":4.69,  
  "stock":94,  
  "brand":"Apple",  
  "category":"smartphones",  
  "thumbnail":"https://i.dummyjson.com/data/products/1/thumbnail.jpg"  
  "images": ["https://i.dummyjson.com/data/products/1/1.jpg"]  
}
```

Gson JSON encoding

```
@JsonClass(generateAdapter = true)
data class Product (
    val brand: String,
    val category: String,
    val description: String,
    val discountPercentage: Double,
    val id: Int,
    val images: List<String>,
    val price: Int,
    val rating: Double,
    val stock: Int,
    val thumbnail: String,
    val title: String
)
```

Set up Retrofit and Gson

```
private val gson = GsonBuilder().serializeNulls().create()

private val httpLoggingInterceptor = HttpLoggingInterceptor()
    .setLevel(HttpLoggingInterceptor.Level.BODY)
private val okHttpClient = OkHttpClient.Builder()
    .addInterceptor(httpLoggingInterceptor)
    .build()

val retrofit = Retrofit.Builder()
    .baseUrl(BASE_URL)
    .addConverterFactory(GsonConverterFactory.create(gson))
    .client(okHttpClient)
    .build()
```

Set up Retrofit and Gson cont'd

```
object API {  
    val retrofitService : SimpleService by lazy {  
        retrofit.create(SimpleService::class.java)  
    }  
}
```

Use Retrofit with coroutines

Launch a new coroutine in the view model:

```
viewModelScope.launch {  
    Log.d("products", API.retrofitService.searchProducts("query"))  
}
```

Display images

Glide

- Third-party image-loading library in Android
- Focused on performance for smoother scrolling
- Supports images, video stills, and animated GIFs

Add Gradle dependency

```
implementation "com.github.bumptech.glide:glide:$glide_version"
```


Add Gradle dependency (New)

```
[versions]
```

```
glide = "4.16.0"
```

libs.versions.toml (Version Catalog)

```
[libraries]
```

```
glide = { group = "com.github.bumptech.glide",  
          name = "glide", version.ref = "glide" }
```

```
dependencies {
```

```
    implementation(libs.glide)
```

```
}
```

build.gradle.kts (Module: app)

Load an image

```
Glide.with(context)
    .load(url)
    .into(imageView);
```

Customize a request with RequestOptions

- Apply a crop to an image
- Apply transitions
- Set options for placeholder image or error image
- Set caching policies

RequestOptions example

```
Glide.with(imgView)
    .load(imgUri)
    .apply(RequestOptions()
        .placeholder(R.drawable.loading_animation)
        .error(R.drawable.ic_broken_image))
    .into(imgView)
```

Demo

Assignment

Make an application that calls API services and lists all products.

- Each product should be displayed with its thumbnail
- when clicking on one item, navigate to another screen that displays the images into horizontal view
- If there exist no images in the selected item, display **No images available for this product** in TextView
- Implement this task using single activity with multi fragments approach and navigate using NavGraph